

Sistema de Otimização de Rotas Médicas

Solução desenvolvida para o **Tech Challenge Fase 2 — FIAP Pós-Tech 9AIDT**.

Otimiza rotas de entrega de medicamentos e insumos hospitalares utilizando **Algoritmo Genético** com suporte de heurísticas e integração com **LLM**.

Arquitetura do Sistema



Componentes

Arquivo	Responsabilidade
data/delivery_points.py	Modelagem dos pontos de entrega médica (nome, coords, prioridade, tempo)
core/genetic_algorithm.py	GA com fitness (distância + prioridade), crossover OX, mutação, elitismo
core/nearest_neighbor.py	Heurística construtiva para inicialização e baseline
core/two_opt.py	Refinamento local pós-GA por inversão de segmentos
reports/route_report.py	Geração de relatório estruturado com métricas e comparação
llm/prompts.py	Templates de prompts: instruções, relatório gerencial e Q&A
llm/llm_client.py	Adaptador desacoplado para OpenAI e Groq
main.py	Orquestrador do pipeline completo

Decisões de projeto

Représentation génétique

As rotas são representadas como **permutações** dos pontos de entrega.

O hospital base (origem) é fixo e não entra na permutação — é adicionado implicitamente no cálculo do custo para garantir o ciclo fechado.

Função de custo (fitness)

```
custo = distancia_total + penalidade_prioridade
```

- **distancia_total**: soma das distâncias Euclidianas do ciclo fechado.
- **penalidade_prioridade**: pacientes de alta prioridade atendidos tarde na rota recebem penalidade proporcional à posição $\times$ fator de prioridade.

Operadores genéticos

- **Seleção**: roleta ponderada por $1/\text{custo}$ (mais apto = mais chance de reproduzir).
- **Crossover**: Order Crossover (OX) — preserva validade da permutação.
- **Mutação**: troca de posições adjacentes com probabilidade configurável.
- **Elitismo**: melhor indivíduo sempre preservado na geração seguinte.

Heurísticas

- **Nearest Neighbor**: inicializa 15% da população com soluções de qualidade.
- **Two Opt Inversion**: refina a melhor rota pós-GA por inversão de segmentos.

Instalação

```
pip install -r requirements.txt
```

Execução

Sem LLM (modo padrão)

```
cd src
python -m main
```

Com LLM via OpenAI

```
export USE_LLM=true
export OPENAI_API_KEY=sk-...
```

```
python -m main
```

```
$env:USE_LLM = "true"  
$env:OPENAI_API_KEY = "sk-..."  
python -m main
```

Com LLM via Groq (gratuito)

```
export USE_LLM=true  
export LLM_PROVIDER=groq  
export GROQ_API_KEY=gsk_...  
python -m main
```

```
$env:USE_LLM = "true"  
$env:LLM_PROVIDER = "groq"  
$env:GROQ_API_KEY = "gsk_..."  
python -m main
```

Funcionalidades LLM

Quando `USE_LLM=true`, o sistema:

1. **Gera instruções operacionais** para a equipe de entrega com base na rota otimizada.
2. **Gera relatório gerencial** com indicadores de eficiência e sugestões de melhoria.
3. **Responde perguntas** em linguagem natural sobre a rota (modo interativo).

Os prompts são estruturados para que o modelo responda **apenas com base nos dados da rota**, evitando alucinações.

Configuração dos pontos de entrega

Edite `data/delivery_points.py` para adicionar os pontos reais:

```
PontoEntrega(  
    nome="UBS Vila Nova",  
    coords=(741, 72),      # coordenadas (x, y)  
    prioridade=1,          # 1=Alta, 2=Média, 3=Baixa  
    tempo_atendimento=15,  # minutos no local  
)
```